

Resumen – Aprendizaje por Refuerzo

Inteligencia Artificial y Neurociencias
Universidad Torcuato Di Tella

Clase 0 – Introducción al Aprendizaje por Refuerzo

El **Aprendizaje por Refuerzo** (RL) es un paradigma en el que un **agente** aprende a comportarse interactuando con un **ambiente**: ejecuta **acciones**, observa el nuevo estado del ambiente y recibe **recompensas**. El objetivo es aprender a actuar de modo de maximizar la recompensa acumulada.

Tres características distintivas

- **Prueba y error**: nadie le dice al agente cuál es la acción correcta; la descubre probando y midiendo consecuencias.
- **Recompensas demoradas**: una acción puede tener efecto sobre recompensas futuras, no solo la inmediata (el problema de *asignación de crédito*).
- **Balance exploración/explotación**: el agente debe *explotar* lo que ya sabe que da buenas recompensas, pero también *explorar* para descubrir mejores opciones.

RL vs. otros paradigmas

- **Aprendizaje supervisado**: aprende de ejemplos etiquetados (entrada $\rightarrow$ salida correcta). En RL no hay etiquetas, solo señales de recompensa.
- **Aprendizaje no supervisado**: busca estructura oculta en datos sin etiquetar. RL no busca estructura, busca *maximizar recompensa*.

Bibliografía de referencia: Sutton & Barto, *Reinforcement Learning: An Introduction* (2018).

Clase 1 – Bandidos de k brazos

El problema de **bandidos de k brazos** es la versión más simple de RL: hay un único estado y k acciones posibles. En cada paso el agente elige una acción a y recibe una recompensa. Cada elección es **independiente** de las anteriores (no hay estados que cambien).

Valor de una acción

El **valor real** de una acción es la recompensa esperada al tomarla:

$$q_*(a) \doteq \mathbb{E}[R_t \mid A_t = a].$$

Como $q_*(a)$ es desconocido, el agente mantiene una **estimación** $Q_t(a)$. El método más simple (*sample-average*) promedia las recompensas obtenidas al tomar a :

$$Q_t(a) = \frac{\text{suma de recompensas al tomar } a \text{ antes de } t}{\text{cantidad de veces que se tomó } a}.$$

Greedy, ε -greedy y exploración

- **Greedy**: elegir siempre $A_t = \arg \max_a Q_t(a)$. Explota, pero nunca explora, y puede quedar atrapado en una acción subóptima.
- **ε -greedy**: con probabilidad $1 - \varepsilon$ elige la acción greedy, con probabilidad ε elige una acción al azar. La exploración queda **distribuida** a lo largo de todo el proceso.

Actualización incremental

En vez de guardar todas las recompensas, se actualiza la estimación paso a paso:

$$Q_{n+1} = Q_n + \frac{1}{n} [R_n - Q_n].$$

Es un caso de la regla general de aprendizaje:

$$\text{NuevaEst} \leftarrow \text{ViejaEst} + \text{TasaApr} [\text{Objetivo} - \text{ViejaEst}].$$

Ventaja: memoria y cómputo $O(1)$ por paso, sin necesidad de almacenar toda la historia.

Problemas no estacionarios

Si $q_*(a)$ cambia con el tiempo, conviene usar un **paso constante** $\alpha \in (0, 1]$ en lugar de $1/n$:

$$Q_{n+1} = Q_n + \alpha [R_n - Q_n].$$

Esto da un **promedio con pesos exponencialmente decrecientes**: da más peso a las recompensas recientes, permitiendo seguir cambios en el ambiente.

Valores iniciales optimistas

Se inicializa $Q_1(a)$ con un valor alto para toda a . Como toda recompensa real “decepciona” frente a la estimación inflada, el agente es empujado a probar las demás acciones: exploración inicial sistemática *sin* aleatoriedad. Limitación: es un mecanismo transitorio, poco útil en problemas no estacionarios.

Clase 2 – Procesos de Decisión de Markov (MDP)

Los **MDP** extienden los bandidos incorporando **estados** $s \in \mathcal{S}$: modelan la **toma de decisiones secuenciales**, donde cada decisión afecta a las siguientes. Ahora se estima:

- $v_*(s)$: el **valor del estado** s .
- $q_*(s, a)$: el **valor de la acción** a en el estado s .

Definición formal

Un MDP es una tupla $(\mathcal{S}, \mathcal{A}, \mathcal{R}, p)$, con estados, acciones y recompensas (conjuntos finitos), y p la dinámica. El tiempo es discreto: $t = 0, 1, 2, \dots$. Una **trayectoria** es una secuencia $S_0, A_0, R_1, S_1, A_1, R_2, \dots$.

La dinámica del ambiente queda caracterizada por:

$$p(s', r \mid s, a) \doteq \Pr\{S_t = s', R_t = r \mid S_{t-1} = s, A_{t-1} = a\},$$

con $\sum_{s'} \sum_r p(s', r \mid s, a) = 1$. Si el ambiente es **determinístico**, esta probabilidad vale 1 para un único (s', r) ; en general es **estocástico**.

Propiedad de Markov

La probabilidad de (S_t, R_t) depende **solo** de (S_{t-1}, A_{t-1}) , no de la historia previa. Esto restringe cómo definimos los estados: deben incluir toda la información del pasado relevante para el futuro.

Retornos y episodios

El objetivo del agente es maximizar el **retorno** (ganancia acumulada esperada):

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=t+1}^T \gamma^{k-t-1} R_k,$$

donde $\gamma \in [0, 1]$ es el **factor de descuento**. Forma recursiva: $G_t = R_{t+1} + \gamma G_{t+1}$. En tareas **episódicas** hay un estado terminal (T finito); en tareas **continuas** $T = \infty$.

Factor de descuento γ : es un parámetro del agente que fija cuánto valora recompensas cercanas vs. lejanas. Con $\gamma = 0$ el agente es *miope* (solo cuenta la recompensa inmediata); con $\gamma \rightarrow 1$ valora más las recompensas lejanas. *Analogía*: una tasa de inflación alta actúa como γ bajo, licuando recompensas futuras.

Políticas y funciones de valor

Una **política** $\pi(a \mid s)$ es la probabilidad de ejecutar a en s (puede ser determinística o estocástica).

$$\begin{aligned} v_\pi(s) &\doteq \mathbb{E}_\pi[G_t \mid S_t = s] && \text{(valor del estado)} \\ q_\pi(s, a) &\doteq \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] && \text{(valor de la acción)} \end{aligned}$$

Propiedad: $q_\pi(s, a) = \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = a]$.

Políticas y valores óptimos

$\pi \geq \pi'$ sii $v_\pi(s) \geq v_{\pi'}(s)$ para todo s . Una política π_* es **óptima** si domina a toda otra. Las **funciones de valor óptimas** son:

$$v_*(s) \doteq \max_{\pi} v_\pi(s), \quad q_*(s, a) \doteq \max_{\pi} q_\pi(s, a).$$

Búsqueda de políticas óptimas

v_* y q_* son desconocidas; el agente las estima.

- **Métodos *model-based*:** si conociéramos $p(s', r | s, a)$ podríamos usar programación dinámica (*Value Iteration*). Pero casi nunca conocemos p : interés teórico.
- **Métodos *model-free*:** no necesitan p ; aprenden interactuando. Ejemplos: Monte Carlo y Q-Learning.

Clase 3 – Métodos Monte Carlo

Los métodos **Monte Carlo** (MC) obtienen soluciones mediante **pruebas aleatorias repetidas**. En RL se basan en **generar episodios, observar y aprender de lo ocurrido**. Como no se conoce $p(s', r | s, a)$, el agente aprende con interacciones **reales** con el ambiente.

Algoritmo MC para estimar π_*

1. Inicializar π al azar, $Q(s, a)$ arbitrario, y Retornos(s, a) como listas vacías.
2. Generar un episodio completo siguiendo π : $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$.
3. Para cada paso t : calcular $G_t = \sum_{k=t+1}^T \gamma^{k-t-1} R_k$, agregarlo a Retornos(S_t, A_t), y actualizar $Q(S_t, A_t) \leftarrow \text{promedio}(\text{Retornos}(S_t, A_t))$.
4. Mejorar la política de forma greedy: $\pi(S_t) \leftarrow \arg \max_a Q(S_t, a)$.

$Q(s, a)$ converge a $q_*(s, a)$ a medida que la cantidad de visitas a (s, a) tiende a infinito.

Problema de exploración y versión final

Con una política puramente greedy, muchos pares (s, a) se visitarían poquísimo (o nunca), y sus estimaciones serían malas. Solución: usar una política **ε -greedy** que garantice explorar todos los pares:

$$\pi(a | s) = \begin{cases} 1 - \varepsilon + \frac{\varepsilon}{|\mathcal{A}(s)|} & \text{si } a = A^* \\ \frac{\varepsilon}{|\mathcal{A}(s)|} & \text{si } a \neq A^* \end{cases}$$

donde $A^* = \arg \max_a Q(s, a)$. La versión final devuelve una política estocástica $\pi \approx \pi_*$.

Clase 4 – Diferencias Temporales: SARSA y Q-learning

MC vs. TD

Ambos parten de la regla $\text{NuevaEst} \leftarrow \text{ViejaEst} + \text{TasaApr} [\text{Objetivo} - \text{ViejaEst}]$, pero difieren en **cuándo** y con qué objetivo actualizan:

- **Monte Carlo:** espera al **final del episodio** y usa el retorno real G_t :

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [G_t - Q(S_t, A_t)].$$

- **Diferencias Temporales (TD):** actualiza **tras cada paso**, sin esperar el final, usando $R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$ como objetivo:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)].$$

Bootstrapping: estimar un valor a partir de *otras estimaciones* propias ($Q(S_{t+1}, \cdot)$) en vez de esperar el resultado real. TD hace bootstrapping; MC no. Ventaja de TD: aprende “sobre la marcha”, útil en tareas continuas o de episodios muy largos.

SARSA (on-policy)

Estima π_* usando **la misma política** ε -greedy para elegir la acción siguiente A' y para estimar el retorno. La actualización usa el par real (S', A') que efectivamente se va a tomar:

$$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)].$$

Por usar la política real en el objetivo, es un método **on-policy**.

Q-learning (off-policy)

Cambia el objetivo por el máximo sobre las acciones, desacoplando la estimación del retorno de la política seguida:

$$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)].$$

Asume una política greedy pura para estimar el retorno, sin importar qué acción se tome de verdad (que sigue siendo ε -greedy). Por eso es un método **off-policy**. La tasa $\alpha \in (0, 1]$ controla la relevancia de la información reciente (útil en ambientes no estacionarios).

Clase 5 – Deep Q-Learning (DQL)

Límite de los métodos tabulares

Los métodos tabulares guardan un valor por cada par (s, a) en una tabla Q . Esto es imposible cuando $\mathcal{S}$ es enorme (e.g. los píxeles de una pantalla de Atari): la cantidad de estados es combinatoriamente gigantesca y, además, dos pantallas casi idénticas cuentan como estados distintos (no hay **generalización**).

Aproximación de la función de valor

Se reemplaza la tabla por una función parametrizada $\hat{q}(s, a, \mathbf{w})$ (e.g. una red neuronal con pesos $\mathbf{w} \in \mathbb{R}^d$, con $d \ll |\mathcal{S}|$), que permite generalizar a estados nunca vistos. Se define el error cuadrático:

$$\overline{\text{QE}}(\mathbf{w}) \doteq \sum_s \sum_a [q_*(s, a) - \hat{q}(s, a, \mathbf{w})]^2.$$

Como $\hat{q}$ es diferenciable respecto de $\mathbf{w}$, se ajusta con **descenso por gradiente estocástico** (SGD). Combinando SGD con bootstrapping ($q_*(S_t, A_t) \approx R_{t+1} + \gamma \max_a \hat{q}(S_{t+1}, a, \mathbf{w})$) se obtiene la regla **semi-gradiente**:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha [R + \gamma \max_a \hat{q}(S', a, \mathbf{w}) - \hat{q}(S, A, \mathbf{w})] \nabla \hat{q}(S, A, \mathbf{w}).$$

No hay garantía de convergencia (a diferencia del caso tabular), pero se usa mucho en la práctica.

Detalles de implementación (Mnih et al. 2013, 2015)

El paper *Playing Atari with Deep RL / Human-level control* aprendió a jugar Atari desde los píxeles con una red convolucional. Dos técnicas clave para estabilizar el entrenamiento:

- **Replay de experiencias:** cada transición $e_t = (S_t, A_t, R_{t+1}, S_{t+1})$ se guarda en una memoria D ; cada actualización de $\mathbf{w}$ se hace sobre un *minibatch* muestreado al azar. Esto rompe la correlación temporal entre muestras consecutivas.
- **Target network:** una copia de la red (*Q-network*) usada para computar el objetivo, que se actualiza solo cada cierta cantidad de iteraciones. Mantener el objetivo “fijo” un rato evita perseguir un blanco que se mueve con cada paso.

El tamaño de la memoria, el del minibatch y la frecuencia de actualización de la target network son **hiperparámetros** (además de α , ε y γ).

Clase 6 – Temas Avanzados

Métodos de gradiente de políticas

En vez de aproximar $q_*(s, a)$ y derivar una política, se aprende **directamente** una política parametrizada $\pi(a \mid s, \theta)$. Útil cuando hay demasiadas acciones o acciones *continuas* (redes eléctricas, trading, robótica, LLMs). Con una métrica de desempeño $J(\theta)$ se hace ascenso por gradiente:

$$\theta_{t+1} = \theta_t + \alpha \nabla J(\theta_t).$$

REINFORCE (gradiente de políticas tipo MC): tras generar un episodio, para cada paso actualiza

$$\theta \leftarrow \theta + \alpha G \nabla \ln \pi(A_t \mid S_t, \theta).$$

Ajusta directamente la probabilidad de elegir A_t : sube o baja proporcionalmente al retorno observado G .

Actor-Critic: variante TD de REINFORCE. Reemplaza G_t por el **error TD** usando bootstrapping:

$$\theta_{t+1} \doteq \theta_t + \alpha (R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}) - \hat{v}(S_t, \mathbf{w})) \nabla \ln \pi(A_t \mid S_t, \theta).$$

La red π es el **actor** (elige acciones) y $\hat{v}$ el **crítico** (evalúa estados). **PPO** (Proximal Policy Optimization) extiende Actor-Critic restringiendo cuánto puede cambiar π por update: más estable, estado del arte en muchos dominios.

Monte Carlo Tree Search (MCTS)

Para juegos como Go, la planificación clásica (Minimax) no alcanza: ~ 250 movidas por posición y sin buenas heurísticas para podar. Idea: usar **trayectorias simuladas** desde el estado actual hasta el final, eligiendo acciones según una política y aproximando el ambiente con un modelo (el agente *imagina*, no interactúa con el ambiente real).

Métodos Rollout: simular trayectorias desde cada acción de S , actualizar solo (S, A) , y luego elegir la acción de mayor valor estimado. La *política rollout* es rápida (p.ej. aleatoria). No se almacenan las mejoras: no es aprendizaje real.

MCTS sí almacena las estimaciones MC en un **árbol** con raíz en S , que se expande incrementalmente. Cuatro etapas por iteración:

1. **Selección:** bajar por el árbol siguiendo la *política del árbol* (balancea exploración/explotación, p.ej. ε -greedy) hasta una hoja.
2. **Expansión:** agregar uno o más hijos a la hoja.
3. **Simulación:** completar la trayectoria con la *política rollout*.
4. **Backup:** propagar el retorno para actualizar los valores (s, a) del árbol.

Tras cierto tiempo, se elige en la raíz la acción de mayor valor. Parte del árbol se reusa; el resto se descarta.

AlphaGo

- **AlphaGo** (2015–2016): MCTS + redes neuronales profundas (*rollout policy, policy network, value network*), entrenadas con 30M de partidas humanas + gradiente de políticas. Venció a Fan Hui (5–0) y a Lee Sedol (4–1).
- **AlphaGo Zero** (2017): MCTS (sin rollouts) + una única red (policy+value) entrenada por **self-play**, *sin* dataset de partidas humanas. Aprendió desde **tabula rasa**, venció 100–0 a AlphaGo y descubrió estrategias nuevas.

Preguntas para las Neurociencias

El RL abre preguntas sobre el cerebro: ¿cómo eligen los animales entre opciones con recompensas desconocidas? ¿Hay un mecanismo parecido a ε -greedy? ¿Cómo se relaciona la recompensa de RL con la **dopamina**? ¿Representa el cerebro algo como una tabla Q o una función de valor? ¿Existe un análogo al *replay de experiencias* (consolidación de memoria durante el sueño, rol del hipocampo en la planificación con rollouts)? ¿Hay estructuras análogas al actor y al crítico?